

Policy Optimisation in RLHF

Stage 3: maximise reward model score while penalising KL divergence from SFT baseline

10 min

Duration

B3 – Apply

Bloom's

L3 – Advanced

Level

AI-RL-IN-TH-000003

ID

The RLHF Optimisation Objective

Stage 3 goal: find policy π_θ that maximises reward without drifting from π_{SFT}

$$\max_{\pi_\theta} \mathbb{E}_{\{x \sim D, y \sim \pi_\theta(\cdot|x)\}} [r(x,y)] - \beta \cdot \text{KL}(\pi_\theta(\cdot|x) \parallel \pi_{\text{SFT}}(\cdot|x))$$

Term 1: Reward maximisation

$r(x,y)$ — the reward model scores each generated response. We want high scores.

Term 2: KL divergence penalty

$\beta \cdot \text{KL}(\pi_\theta \parallel \pi_{\text{SFT}})$ — penalises divergence from the SFT policy. Prevents reward hacking.

β is the KL coefficient — key hyperparameter:

$\beta \rightarrow 0$: pure reward maximisation (reward hacking risk)

$\beta \rightarrow \infty$: policy stays = π_{SFT} (no improvement)

Typical $\beta = 0.01 - 0.1$ in practice (requires tuning)

Token-Level Reward Structure — How Reward Is Applied

Simplified View (End-of-Sequence)

- Reward signal applied at the final token only
- $r(x,y)$ is a scalar for the whole response
- All tokens in the response share the same reward signal
- Sparse: most tokens receive zero reward during training
- This is the original InstructGPT approach

KL at Token Level (Per-Step)

- KL divergence computed per token during generation
- $KL(\pi_{\theta}(a_t | s_t) \parallel \pi_{SFT}(a_t | s_t))$ for each token t
- Provides a dense penalty signal per step
- Prevents individual token distributions from drifting
- Standard implementation in TRL PPOTrainer

The Role of the Reference Model (π_{SFT})

What the Reference Model Does

Provides the KL baseline. For each token t at each step, we compute $\text{KL}(\pi_{\theta}(t) \parallel \pi_{\text{SFT}}(t))$ to measure drift.

Why It's Frozen

If we updated π_{SFT} during training, the KL penalty would always be zero — no constraint on policy drift.

Memory Impact

π_{SFT} occupies the same VRAM as the actor. In total: 4 model copies needed (actor + critic + reference + RM).

Practical Note

In TRL's PPOTrainer, the reference model is loaded automatically from the SFT checkpoint. Pass `create_reference_model=True`.

KEY REFERENCES

- Ouyang, L. et al. (2022). InstructGPT. NeurIPS 2022.
- Schulman, J. et al. (2017). Proximal Policy Optimisation. arXiv:1707.06347
- Lambert, N. (2024). RLHF Book. rlhfbook.com



Next: Concept 22: PPO — Proximal Policy Optimisation (AI-RL-IN-TH-000004)